
Module-1

Introduction to C

4.1 Introduction

- ✓ C is a high-level programming language.
- ✓ The programming language C was developed in the early **1970s** by **Dennis Ritchie** at **Bell Laboratories** to be used by the UNIX operating system.
- ✓ It was named C because many of its features were derived from an earlier language called B.
- ✓ Although C was designed for implementing system software, it was later on widely used for developing **portable application software**.
- ✓ C is one of the most popular programming languages.
- ✓ It is being used on several different software platforms.
- ✓ In a nutshell, there are a few computer architectures for which a C compiler does not exist.
- ✓ It is a good idea to learn C because few other programming languages such as C++ and Java are also based on C which means you will be able to learn them more easily in the future.

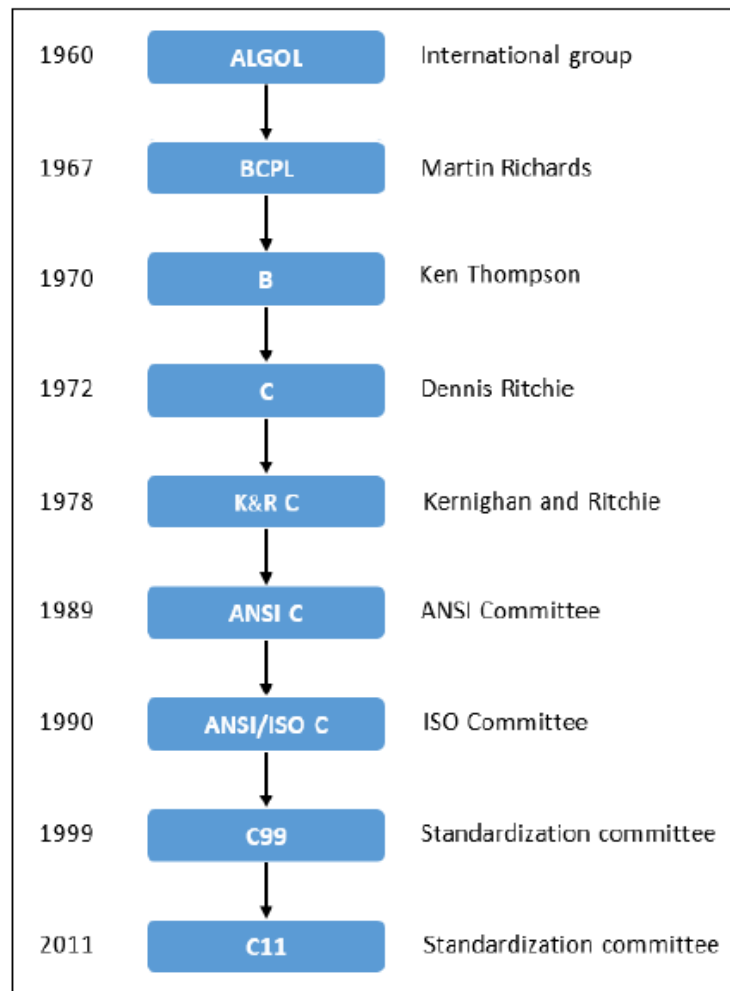


Figure 4.1. Taxonomy of C Language

4.2 Characteristics of C

C is a robust language whose rich set of **built-in functions and operators** can be used to write complex programs. The C compiler combines the features of assembly languages and high-level

languages, which makes it best suited for writing system software as well as business packages. Some basic characteristics of C language that define the language and have led to its popularity as a programming language are listed below:

- ✓ C is a **high-level programming language**, which enables the programmer to concentrate on the problem at hand and not worry about the machine code on which the program would be run.
- ✓ **Small size**: C has only 32 keywords. This makes it relatively easy to learn as compared to other languages.
- ✓ C makes extensive use of **function calls**.
- ✓ C is well suited for **structured programming**. In this programming approach, C enables users to think of a problem in terms of functions/modules where the collection of all the modules makes up a complete program. This feature facilitates ease in program debugging, testing, and maintenance.
- ✓ Unlike PASCAL it supports **loose typing** (as a character can be treated as an integer and vice versa).
- ✓ **Structured language** as the code can be organized as a collection of one or more functions
- ✓ **Stable language**: ANSI C was created in 1983 and since then it has not been revised.
- ✓ **Quick language** as a well written C program is likely to be as quick as or quicker than a program written in any other language.
- ✓ Facilitates **low-level programming**.
- ✓ **Core language**: C is a core language as many other programming languages (like C++, Java, Perl, etc) are based on C.
- ✓ C is a **portable language**, i.e., a C program written on one computer can be run on another computer with little or no modification.
- ✓ C is an **extensible language** as it enables the user his own functions to the C library.
- ✓ C is often treated as the second best language for any given programming task.

4.3 Uses of C

C is a very simple language that is widely used by software professionals around the globe. The uses of C language can be summarized as follows:

- ✓ C is a very simple language that is widely used by **software professionals** around the globe.
- ✓ C is primarily used for **system programming**. The portability, efficiency, the ability to access specific hardware addresses and low runtime.
- ✓ The **compiler, libraries and interpreters** of other programming languages are often implemented in C.

-
- ✓ For portability and convenience reasons, C is sometimes used as an intermediate language for implementation of other languages. Major parts of popular **operating systems like windows, UNIX, Linux** are still written in C.
 - ✓ C is widely used to implement **end-user applications**.
 - ✓ Mobile devices like cellular phones and palmtops consisting of **microprocessor, operating system and some applications** are written in C.
 - ✓ Common consumer devices like microwave ovens, washing machines and digital cameras are consisting of many **programs** which are written in C language.
 - ✓ Several professional **3D computer games and many popular gaming frameworks** have been built using C language.

4.4 Basic Concepts of a C Program or Structure of a C Program

- ✓ The basic concepts of a C program can be explained by writing the structure of a C program.
- ✓ The structure of a C program is nothing but the rules that are to be followed while writing a C program.
- ✓ **The structure of a C program** is shown below:

```
[Comments]
[Preprocessor Directives]
[Global Declarations]
[Function Declarations]
[Function Definitions]
main()
{
    [Declaration Section]
    [Executable Section]
}
```

1. Comments

- ✓ **At the beginning of each program is a comment with a short description of the problem to be solved.**
- ✓ **We can use the comments anywhere in the program.**
- ✓ The comments section is **optional**.
Ex: 1. /* Program1: To find the sum of two numbers*/
2. // Program2: To calculate the area and perimeter of circle
- ✓ **The symbol /* and ends with */ represents the multiline comment.**
- ✓ **The symbol // can also be used for representing single line comment.**

2. Preprocessor directives

- ✓ **The preprocessor statements start with # symbol.**
- ✓ **These statements instruct the compiler to include some of the files in the beginning of the program.**
Ex: #include<stdio.h>
#include<math.h>
are the files that the compiler includes in the beginning of the program.
- ✓ **The line containing #include<stdio.h> tells the compiler to allow our program to perform standard input and output 'stdio' operations.**

-
- ✓ **The '#include' directive tells the compiler that we will be using parts of the standard function library.**
 - ✓ Information about these functions is contained in a series of 'header files' (stdio.h).
 - ✓ .h says this file is a header file.
 - ✓ The pointed brackets < and > tell the compiler the exact location of this header file.
 - ✓ Using the preprocessor directives the user can define the constants also.
Ex: #define PI 3.142

3. Global Declarations

- ✓ The variables that are declared above (before) the main program are called global variables.
- ✓ The global variables can be accessed anywhere in the main program and in all the other functions.

4. Function declarations and Definitions

- ✓ In this section the functions are declared.
- ✓ Immediately after the functions are declared, the functions can be defined.

5. The program header: main()

- ✓ **Every program must have a main function.**
- ✓ **Always the C program begins its execution from main.**

6. Body of the program

- ✓ **After the header or top lines is a set of braces ({ and }) containing a series of 'C' statements which comprise the 'body'- this is called the action portion of the program.**

Ex: #include<stdio.h>

```
main()
{
    /* action portion of the program*/
}
```

- ✓ The body of the program contains two essential parts:
 1. Declaration section
 2. Executable section

7. Declaration section

- ✓ **The variables that are used inside the function should be declared in the declaration section.**
- ✓ For example, consider the declaration shown below:

```
int sum=0;
int a;
float b;
```

Here, the variable sum is declared as an integer variable and it is initialized to zero. The variable a is declared as an integer variable whereas the variable b is declared as a floating point variable.

8. Executable section

- ✓ **They represent the instructions given to the computer to perform a specific task.**

- ✓ The instructions can be input/output statements, expressions to be evaluated, simple assignment statements, control statements such as if statement, for statement etc.
- ✓ Each executable statement ends with “;”.

Example: Write a C program to display “Hello World”.

```
#include<stdio.h>
void main()
{
    printf(“Hello World”);
}
```

4.5 Files used in a C Program

A C program uses four types of files as follows:

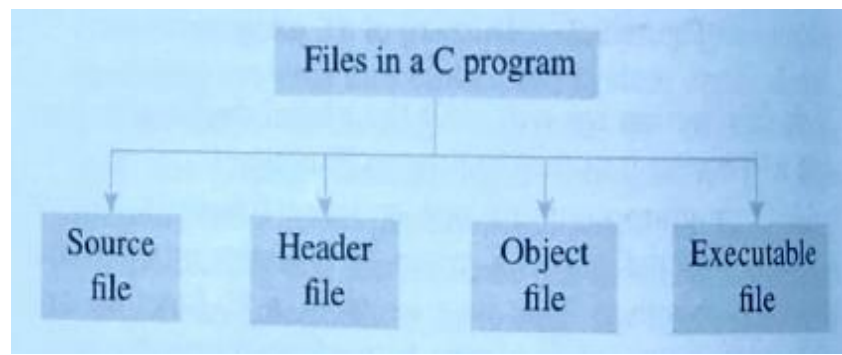


Figure 4.2. Files in a C program

1. Source Code File

- ✓ This file includes the **source code of the program**.
- ✓ The **extension** for these kind of files are '.c'.
- ✓ It defines the **main and many more functions** written in C.
- ✓ **main()** is the starting point of the program. It may also contain **other source code files**.

2. Header Files

- ✓ They have an **extension '.h'**.
- ✓ They contain the **C function declarations and macro definitions** that are shared between various source files.

Advantages of header files:

1. At times the programmer may want to use the same **subroutines for different programs**. To do this, he would just **compile the code of the subroutine once and link to the resulting object file in any file** in which the functionalities of this subroutine are required.
2. At times the **programmer may want to change or add the subroutines and reflect those changes in all the programs**. For doing this, he will have to only change the source file for the subroutines, recompile the source code and then recompile and re-link the program. This tells us that including a header file will make it easier at all levels of the program. If we need to modify anything then changes are made only in the subroutines after which all the changes will be reflected.

Standard header files

- ✓ C provides us with some standard header files which are available easily.

Common standard header files are:

- ✓ **string.h** – used for handling string functions.
 - ✓ **string.h** – used for handling string functions.
 - ✓ **stdlib.h** – used for some miscellaneous functions.
 - ✓ **stdio.h** – used for giving standardized input and output.
 - ✓ **math.h** – used for mathematical functions.
 - ✓ **alloc.h** – used for dynamic memory allocation.
 - ✓ **conio.h** – used for clearing the screen.
-
- ✓ The header files are added at the **start of the source code** so that they can be used by more than one function of the same file.

3. Object files

- ✓ They are the files that are generated by the **compiler as the source code file is processed**.
- ✓ These files generally contain the **binary code of the function definitions**.
- ✓ The object file is used by the **linker for producing an executable file** for combining the object files together. It has a **'o' extension**.

4. Executable file

- ✓ This file is generated by the **linker**.
- ✓ **Various object files are linked by the linker** for producing a **binary file** which will be executed directly.
- ✓ They have an **'exe' extension**.

4.6 Compiling and Executing C Programs

- ✓ C is a **compiled language**.
- ✓ The programming process starts with creating a **source file** that consists of the statements of the program written in C language. This source file usually contains ASCII characters and can be produced with a text editor, such as Windows notepad, or in an Integrated Design Environment.
- ✓ The source file is then processed by a special program called a **compiler**.
- ✓ **The compiler translates the source code into an object code.**
- ✓ The object code contains the **machine instructions** for the CPU, and calls to the operating system API (Application Programming Interface).
- ✓ However, even the object file is not an executable file. Therefore, in the next step, **the object file is processed with another special program called a linker**.
- ✓ While there is a different compiler for every individual language, the same linker is used for object files regardless of the original language in which the new program was written. The output of the linker is an executable or runnable file. The process is shown in Figure 4.3.

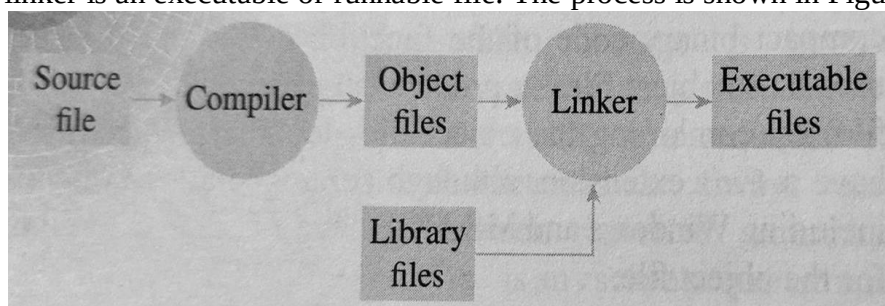


Figure 4.3. Overview of compilation and execution process

- ✓ In C language programs, there are two kinds of source files. In addition to the **main (.c) source file**, which contains executable statements there are also **header (.h) source files**.
- ✓ Every C program uses standard header files, which are written as part of the source code for **modular C programs**.
- ✓ The compilation process shown in Figure 4.4 is done in two steps. In the first step, the **preprocessor program** reads the source file as text, and produces another text file as output. The output of the preprocessor is a text file which does not contain any preprocessor statements. This file is ready to be processed by the compiler.

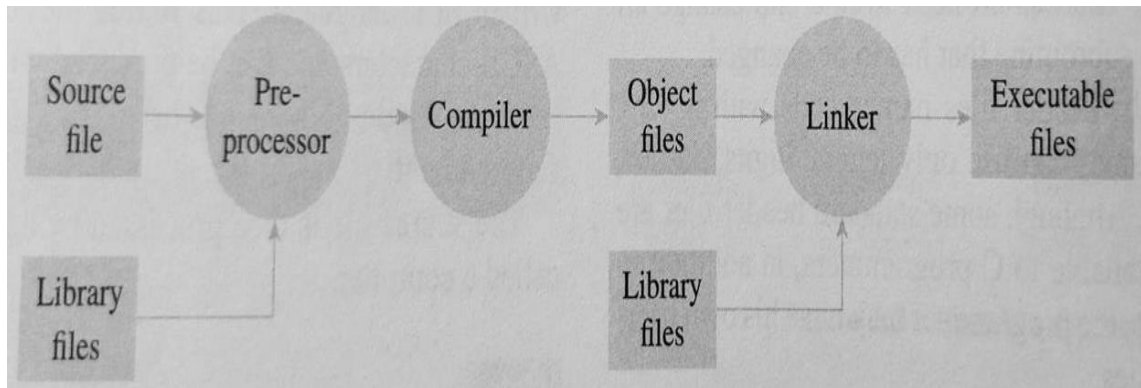


Figure 4.4. Preprocessing before compilation

- ✓ **The linker combines the object file with library routines (supplied with the compiler) to produce the final executable file.**
- ✓ In modular programming, the source code is divided into two or more source files. All these source files are compiled separately thereby producing multiple object files. These object files are combined by the linker to produce an executable file (Figure 4.5).

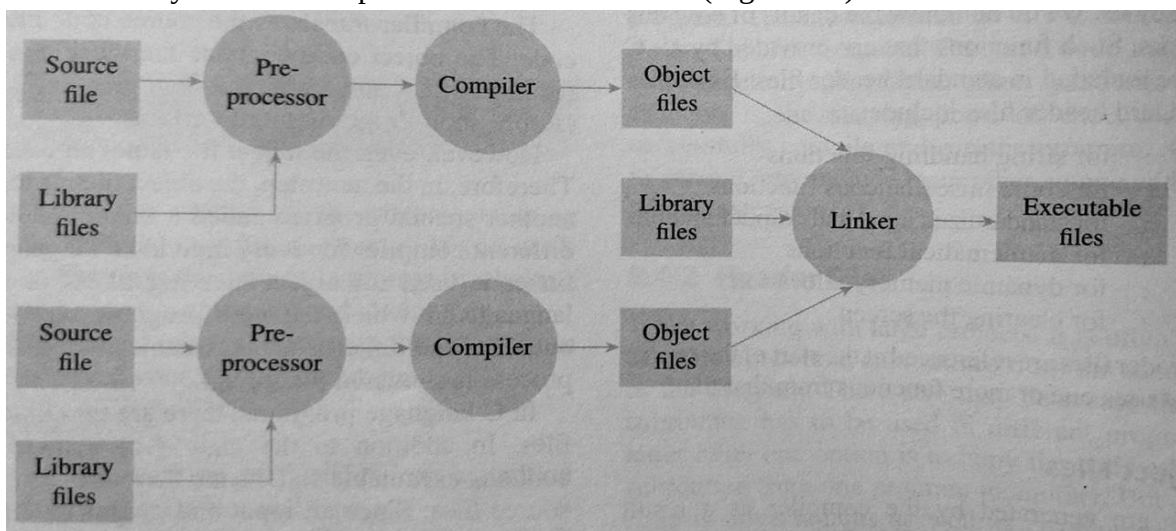


Figure 4.5. Modular programming-the complete compilation and execution process

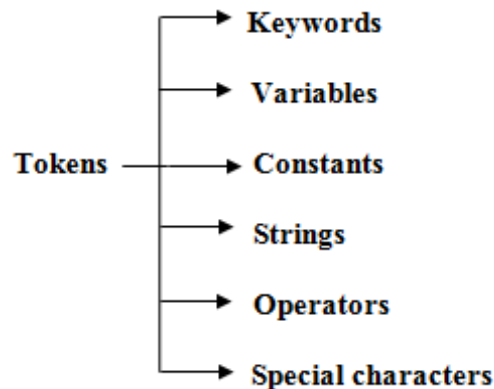
4.7 Using Comments

- ✓ Many a time the meaning or the purpose of the file code is not clear to the reader.
- ✓ Therefore it is a **good programming practice to place some comments in the code to help the reader understand the code clearly.**
- ✓ Comments are just a **way of explaining what a program does.**
- ✓ It is merely an internal program documentation.

- ✓ The **compiler ignores the comments when forming the object file means that the comments are non-executable statements.**
- ✓ **C supports two types of comments.**
// is used to comment a **single statement**. This is known as a **line comment**. A line comment can be placed anywhere on the line and it does not require to be specifically ended as the end of the line automatically ends the line.
/* is used to comment **multiple statements**. A /* is ended with */ and all statements that lie within these characters are commented. This type of comment is known as block comment.

4.8 C Tokens

- ✓ **Tokens are the basic building blocks in C language.**
- ✓ **A token is the smallest individual unit in a C program.**
- ✓ This means that a program is constructed using a combination of these tokens.
- ✓ There are six main types of tokens in C as shown below:



1. Keywords:

- ✓ **Keywords are the tokens which have predefined meaning in C language, whose meaning cannot be changed by the user.**
- ✓ All keywords are basically a **sequence of characters** that have a **fixed meaning**.
- ✓ They are also called **reserved** words.
- ✓ C Keywords are case sensitive. Therefore all C keywords must be written in small letters.
int, float, if, while, void etc are valid keywords.
Int, Float, IF, VOID etc are invalid keywords. (They are written using capital letters).

List of keywords:

- ✓ There are total 32 keywords in C language as shown below.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

2. Variables: A variable is a **data item whose value changes during the execution of program.**

Ex: int a,b,sum;

3. Constants: A constant is a **data item which will not change during the execution of a program.**

Ex: #define PI 3.142

4. Strings: String is an **array of characters and terminated by NULL character** which is denoted by **'\0'**.

Ex: char name[21];

5. Operators: Operator is a **symbol (or token) that specifies the operation to be performed on various types of data.**

Ex: Arithmetic operators (+,-,*,/), Relational operators (&&,||,!), Logical operators (>,>=,<,<=) and Assignment operator (=) etc.

6. Special characters: [], { }, () etc. used in the program to execute the code correctly and helps to write a complex codes by special symbols.

4.9 CHARACTER SET IN C

- ✓ Like in natural languages, computer languages also use a **character set that defines the fundamental units used to represent information.**
- ✓ In C, a character means any letter from **English alphabet, a digit or a special symbol** used to represent information.
- ✓ These characters when combined together form tokens that act as basic building blocks of a C program.
- ✓ The character set of C can therefore be given as:

a. English alphabet: Include both lower case (a z) as well as upper case (A Z) letters

b. Digits: Include numerical digits from 0 to 9

c. Special characters: Include symbols such as, % &) < > * S /) [" etc.,

d. White space characters: These characters are used to print a blank space on the screen.

They are shown in Figure 4.6.

White space character	Meaning
\b	Blank space
\t	Horizontal tab
\v	Vertical return
\r	Carriage return
\f	Form feed
\n	New line

Figure 4.6. White space characters in C

e. Escape sequence: **'\n'** is an escape sequence and represents a newline character. It is used to print the message on a new line on the screen. Like the newline character, the other escape sequences supported by C language are shown in Table 4.1.

Table 4.1: Escape sequences

Escape sequence	Purpose	Escape sequence	Purpose
\a	Audible signal	\?	Question mark
\b	Backspace	\\	Back slash
\t	Tab	\'	Single quote
\n	Newline	\"	Double quote
\v	Vertical tab	\0	Octal constant
\f	New page\ Clear screen	\x	Hexadecimal constant
\r	Carriage return		

4.10 Identifiers

- ✓ Identifiers help us to **identify data and other objects in the program.**
- ✓ Identifiers are basically the **names given to program elements such as variables, arrays, and functions.**
- ✓ Identifiers may consist of sequence of **letters, numerals, or underscores.**

Rules for Forming Identifier Names

Some rules have to be followed while forming identifier names. They are as follows:

- ✓ Identifiers cannot include any **special characters or punctuation marks (like #, \$, ^, ?, ., etc.) except the underscore ‘_’.**
- ✓ There **cannot be two successive underscores.**
- ✓ **Keywords cannot be used** as identifiers.
- ✓ The **case of alphabetic characters** that form the identifier name is **significant**. For example, ‘FIRST’ is different from ‘first’ and ‘First’.
- ✓ Identifiers must **begin with a letter or an underscore**. However, use of underscore as the first character must be avoided because several compiler-defined identifiers in the standard C library have underscore as their first character.
- ✓ Identifiers can be of any reasonable length. They should **not contain more than 31 characters**. They can actually be longer than 31, but the compiler looks at only the first 31 characters of the name.

Examples of valid identifiers include:

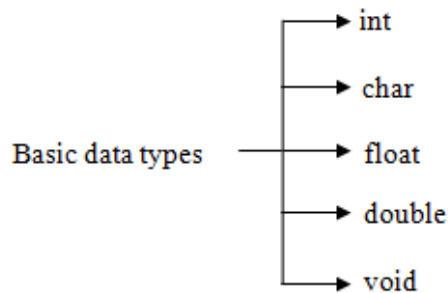
roll_number, marks, name, emp_number, basic_pay, HRA, DA, dept_code, DeptCode, RollNo, EMP_NO

Examples of invalid identifiers include:

23 student, %marks, @name, #emp_number, basic.pay, -HRA, (DA), &dept_code, auto

4.11 Basic Data types in C and Sizes

- ✓ The data type defines the type of data stored in a memory location.
- ✓ The data type determines how much memory should be allocated for a variable.
- ✓ The data types that can be manipulated by machine instructions are called 'basic or primitive data types'.



i. int:

- ✓ An int is a keyword which is used for defining integers in C language.
- ✓ Using int the programmer can inform the compiler that the data associated with this should be treated as integer.
- ✓ Using 'int' compiler determines the size of the data (2 bytes) and reserve space in memory to store the data.
- ✓ Integer data types namely:
 1. short int
 2. int
 3. long int

Type	Size
short int	2 bytes
int	2 bytes
long int	4 bytes

Ex: int a,b,c;

ii. float:

- ✓ A float is a keyword which is used to define floating point numbers in C language.
- ✓ The programmer can inform the compiler that the data associated with this keyword should be treated as floating point number.
- ✓ The default precision of floating point number is 6 digits after dot(.).

	Size of float
16-bit Machine	4 bytes
32-bit Machine	8 bytes

Ex: float x,y,z;

iii. double:

- ✓ It is a keyword which is used to define long floating point numbers in C language.
- ✓ The default precision of floating point number is 14 digits after dot(.).

	Size of double
16-bit Machine	8 bytes
32-bit Machine	16 bytes

Ex: double p,q,r;

iv. char:

- ✓ It is a keyword which is used to define single character or a sequence of characters called String in C language.
- ✓ Using this keyword, the compiler determines the size of the data and reserve space in memory to store the data.
- ✓ Each character stored in the memory is associated with a unique value called an ASCII (American Standards Code for Information Interchange).

	Size of char	Range of Unsigned char	Range of Signed char
16/32-bit Machine	1 byte	0 to 255	-128 to +127

Ex: char ch; // ch variable stores a single character Ex: ch= 'a';
 char s[20]; // s variable stores a string(group of characters) Ex: s= "jtdvg";

v. void:

- ✓ It is an empty data type, since no value is associated with this data type.
- ✓ It does not occupy any space in the memory.

	Size of void	Range
16/32-bit Machine	0	No value

Ex: void main()
 {

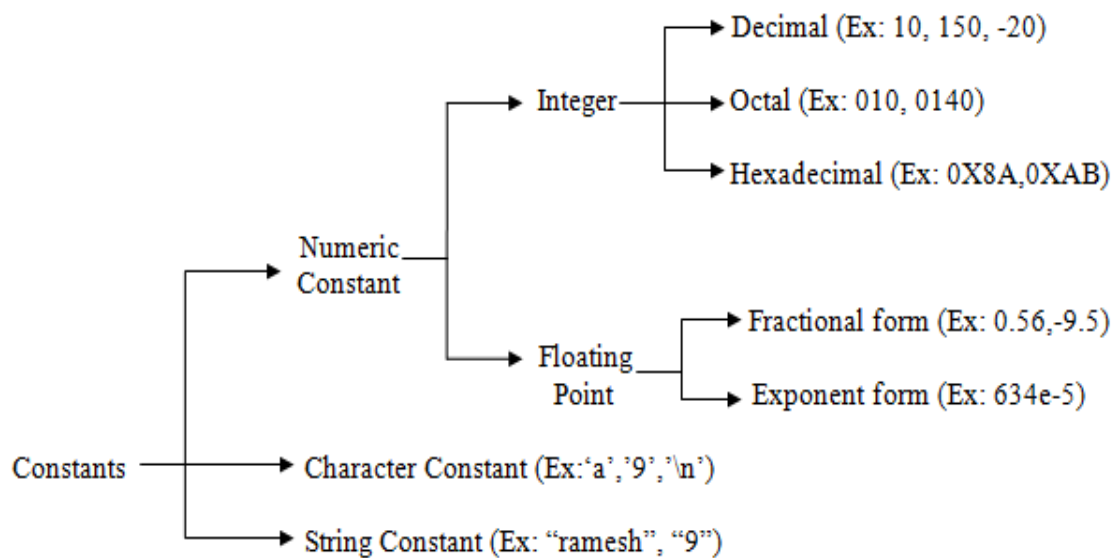
 }

It is primarily used in three cases:

- ✓ To specify the **return type of a function** (when the function returns no value)
- ✓ To specify the **parameters of the function** (when the function accepts no arguments from the caller)
- ✓ To create **generic pointers**.

4.12 Constants

- ✓ A constant is a data item which will not change during the execution of a program.
or
Constants are identifiers whose values do not change.
- ✓ While values of variables can be changed at any time, values of constants can never be changed. Constants are used to define fixed values like mathematical constant pi or the charge on an electron so that their value does not get changed in the program even by mistake.
- ✓ C allows the programmer to specify constants of integer type, floating point type, character type, and string type.
- ✓ The **constants cannot be modified** in the program.



4.12.1 Types of Constants

1. Integer Constants

- ✓ A constant of integer type consists of a sequence of digits.
- ✓ For example, 1, 34, 567, 8907 are valid integer constants.
- ✓ Integer literals can be expressed in **decimal, octal or hexadecimal notation**.
- ✓ By default an integer is expressed in decimal notation. Decimal integers consists of a set of **digits 0 through 9, preceded with an optional + or – sign**. Example: 123, -123, +123, and 0.
- ✓ An integer constant **preceded by a zero (0) is an octal number**. Octal integers consist of a set of digits, **0 through 7**. Example: 012, 0, 01234.
- ✓ An integer constant is expressed in **hexadecimal notation if it is preceded with 0x or 0X**. Hexadecimal numbers contain **digits from 0-9 and letters A through F**, which represent numbers 10 through 15. Example: 0x12, 0x7F, 0xABCD, 0x1A3B.

2. Floating Point Constants

- ✓ A floating point constant consists of an **integer part, a decimal point, a fractional part, and an exponent field containing an e or E** (e means exponent) followed by an integer where the fraction part and integer part are a sequence of digits. However, it is not necessary that every floating point constant must contain all these parts. Some floating point numbers may have certain parts missing. Some valid examples of floating point numbers are: 0.02, -0.23, 123.456, +0.34 123, 0.9, -0.7, +0.8 etc.
- ✓ To make it a **float type literal**, you must specify it using **Suffix F or f**. Consider some valid floating point literals given below. (Note that suffix L is for long double.)
0.02F 0.34f 3.141592654L 0.002146 2.146E-3
- ✓ A floating point number may also be expressed in **scientific notation**. Therefore, the numbers given below are valid floating point numbers:
0.5e2 14E-2 1.2e+3 2.1E-3 -5.6e-2

3. Character Constants

- ✓ A character constant consists of a **single character enclosed in single quotes**.
- ✓ For example, 'a' and '@' are character constants.
- ✓ In computers, characters are stored using machines character set using **ASCII codes**. All escape sequences mentioned in Table 4.1 are also character constants.

4. String Constants

- ✓ A string constant is in **double quotes**.
- ✓ So “a” is not the same as ‘a’.
- ✓ The characters comprising the string constant are stored in **successive memory locations**.
- ✓ When a string constant is encountered in a C program, the compiler records the address of the first character and appends a null character (‘\0’) to the string to mark the end of the string. Thus, length of a string constant is equal to number of characters in the string plus 1 (for the null character). Therefore, the length of string literal “hello” is 6.

4.12.2 Declaring Constants

- ✓ To declare a constant, precede the normal variable declaration with “**const**” keyword and assign it a value. For example,
`const float pi = 3.14;`
The const keyword specifies that the value of pi cannot change.
- ✓ However, another way to designate a constant is to use the pre-processor command **define**.
Example: `#define PI 3.14159`
`#define service_tax 0.12`

In these examples, the value of pi will never change but service tax may change. Whenever the value of the service tax is altered, it needs to be corrected only in the define statement.

Some rules that need to be applied to a #define statement which defines a constant:

Rule 1: Constant names are usually written in capital letters to visually distinguish them from other variable names which are normally written in lower case characters.

Rule 2: No blank spaces are permitted between the # symbol and define keyword.

Rule 3: Blank space must be used between #define and constant name and between constant name and constant value.

Rule 4: #define is a pre-processor compiler directive and not a statement. Therefore, it does not end with a semi-colon.

4.13 Variable

- ✓ A variable is a name given to a memory location within the computer that can hold one value at a time.

OR

A variable is a data item whose value changes during the execution of program.

- ✓ Every variable should be associated with **type, size and value**.
- ✓ Whenever a new value is placed into a variable, it replaces the previous value.

4.13.1 Rules for defining variables

Some rules have to be followed while forming variable names. They are as follows:

- ✓ Variables cannot include any **special characters or punctuation marks (like #, \$, ^, ?, ., etc.) except the underscore ‘_’**.
- ✓ There **cannot be two successive underscores**.
- ✓ **Keywords cannot be used** as variables.

- ✓ The **case of alphabetic characters** that form the variable name is **significant**. For example, 'FIRST' is different from 'first' and 'First'.
- ✓ Variables must **begin with a letter or an underscore**.
- ✓ Variables can be of any reasonable length. They should **not contain more than 31 characters**. They can actually be longer than 31, but the compiler looks at only the first 31 characters of the name.

Ex: Classify the following into valid and invalid variable names in C. If invalid give reasons:

Variable	Valid/Invalid	Reason
principle_amount	Valid	
A	Valid	
Sum1	Valid	
for1	Valid	
if	Invalid	It is a keyword
for	Invalid	It is a keyword
3_factorial	Invalid	Should not start with digit
Sum,1	Invalid	Comma should not be there
sum-of-digits	Invalid	Minus sign should not be there
sum_of_digits	Valid	
sum of digits	Invalid	No spaces are allowed
\$sum	Invalid	\$ sign should not be there
sum=	Invalid	= sign should not be there
one+two	Invalid	+ sign should not be there
sum!	Invalid	! sign should not be there
int	Invalid	It is a keyword
\$roll no	Invalid	\$ sign should not be there
_name1	Valid	
James bond	Invalid	No spaces are allowed

4.13.2 Types of Variables

C language supports two basic kinds of variables numeric and character.

1. Numeric Variables

- ✓ Numeric variables can be used to **store either integer values or floating point values**.
- ✓ While an **integer value is a whole number without a fraction part or decimal point, a floating point value can have a decimal point**.
- ✓ Numeric variables may also be associated with modifiers, such as **short, long, signed, and unsigned**. The difference between signed and unsigned numeric variables is that **signed variables can be either negative or positive but unsigned variables can only be positive**.
- ✓ When we do not specify the signed/ unsigned modifier, **C language automatically takes it as a signed variable**. To declare an unsigned variable, the unsigned modifier must be explicitly added during the declaration of the variable.

2. Character Variables

- ✓ Character variables are just **single characters enclosed within single quotes**. These characters could be any character from the ASCII character set-letters ('a', 'A'), numerals ('2'), or special characters ('&').
- ✓ A number that is given in single quotes is not the same as a number without them. This is because 2 is treated as an integer value, but '2' is considered character not an integer.

4.13.3 Declaring/Defining a Variable

- ✓ Each variable to be used in the program must be declared.
- ✓ 'Declaration' tells the computer which storage locations or variables to use in the program.
- ✓ **It is a method of informing the compiler to reserve the memory space for the program data based on the type of variables.**
- ✓ To declare a variable, specify the data type of the variable followed by its name.
- ✓ The **data type** indicates the **kind of values that the variable will store.**

Syntax:

datatype v₁,v₂,.....,v_n;

Where, datatype: it can be int, float, char, double etc.

v₁,v₂,---,v_n: is a list of variables, which are separated by commas.

Ex: int a,b,c;

float p,t,r,si;

4.13.4 Initializing the variables

- ✓ **Initialization is the process of assigning values to the variables.**

Syntax:

datatype var_name=data;

Where, datatype: is the type of the data to be stored in memory location. (int,float,char,double)

var_name: name of a variable

= is assignment operator

data: is the value to be stored in memory associated with variable var_name.

Ex: int a=10;

4.14 Input/Output statements in C

- ✓ A Stream is the **source of data** as well as the **destination of data.**
- ✓ Streams are associated with a **physical device** such as a **monitor or a file** stored on the secondary memory.
- ✓ C uses two forms of streams:
 - **Text stream:** In a text stream, sequence of characters is divided into lines, each line being terminated with a new character(\n).
 - **Binary stream:** A binary stream contains data values using their memory representation.

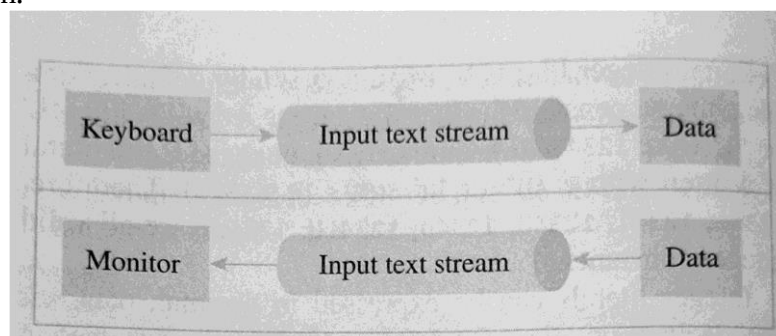


Figure 4.7. Input and output streams in C

Formatting Input/Output

- ✓ C language supports two formatting functions **printf** and **scanf**.
- ✓ **printf** is used to convert data stored in the program into a text stream for output to the monitor and **scanf** is used to convert the text stream coming from the keyboard to data values and stores them in program variables.

1. printf()

- ✓ The printf() function (stands for print formatting), is used to **display information required by the user and also prints the values of variables.**
- ✓ For this, **the printf() function takes data values, converts them to a text stream using formatting specifications in the control string and passes the resulting text stream to standard output.**
- ✓ Each data value to be formatted in the text stream is described using a separate conversion specification in the control string.
- ✓ The **specification in the control string** describes **data value's type, size, specific format.**

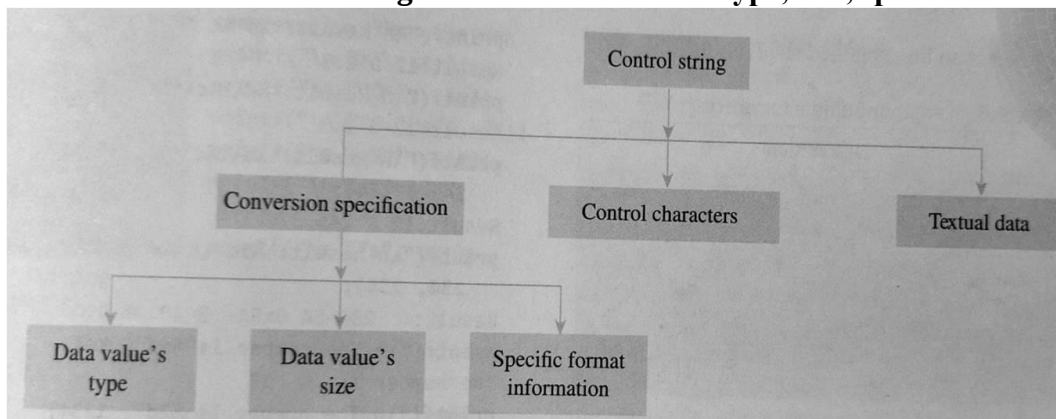


Figure 4.8. printf() in C

The syntax of printf function:

printf("control string", variable list);

- ✓ The function accepts two parameters- **control string & variable list.**
- ✓ The control string may also contain the text to be printed like instructions to the user, captions, identifiers or any other text to make the output readable.
- ✓ In some printf statements there may be only a text string that has to be displayed on the screen.
- ✓ The control characters can also be included in the printf statement like \n, \t, \a, etc.
- ✓ The **prototype of control string:**
%[flags][width][.precision][length modifier] type specifier
- ✓ Each control string must begin with a % sign. The % sign specifies how the next variable in the list of variables has to be printed.
- ✓ After % sign follows:

Flags

- ✓ Flag is an optional argument which specifies output justifications such as numerical sign, trailing zeros or octal, decimal or hexadecimal prefixes.

Types of flags:

-	:	Left justify
+	:	Display data with numeric sign
#	:	Provide additional specifier o,O,X,0,0x
0	:	Left-padding with zeros

Width

- ✓ It is an optional argument which specifies minimum number of positions in the output.

Precision

- ✓ Precision is an optional argument which specifies the maximum number of characters to print.

-
- ✓ Therefore, a conversion specification %7.3f means print a floating point value of maximum 7 digits where 3 digits are allotted for the digits after the decimal point.

Length modifiers

Table 4.2 Length modifiers for printf()

Length	Description
h	When the argument is a short int or unsigned short int
l	When the argument is a long int or unsigned long int (used for integer specifiers)
L	When the argument is a long double (used for floating point specifiers)

Type specifiers

- ✓ Type specifiers are used to define the type and interpretation of the value of the corresponding argument.

Table 4.3 Type specifiers for printf()

Type	Qualifying input
c	For single characters
d	For integer values
f	For floating point numbers
E, e	Floating point numbers in exponential format
G, g	Floating point numbers in the shorter of e format
o	For octal numbers
s	For a sequence of (string of) characters
u	For unsigned integer values
X, x	For hexadecimal values

Guidelines/Rules for printf()

- ✓ A printf() always contains a string or format string in quotation marks.
- ✓ The control string may or may not be followed by some variables or expressions whose value we want printed.
- ✓ Each value to be printed needs a 'conversion specification' like %d to hold its place in the control string.
- ✓ This conversion specification describes the exact way the value is to be printed.
- ✓ When printf() is executed each conversion specification is replaced by the value of the corresponding expression, then print according to the rules in specification.
- ✓ The symbols \n or \t in control string tell the machine to skip to new line or tab. It affects the appearance of the output but not displayed as part of it.
- ✓ A word or blank space or punctuation symbols within the control string will print exactly as it appears.

- ✓ If there are variables or expressions to be printed, commas are used to separate them from the control string and each other, once comma is used as separator, it is not necessary to add blank spaces (not allowed).

Ex: `printf ("%d%d \n",a,b);`
and
`printf ("%d%d \n", a, b);`

Control String or Format Specifiers Newline character Variables to be printed

Both are same

2. scanf()

- ✓ The `scanf()` function stands for **scan formatting** and **is used to read formatted data from the keyboard**.
- ✓ **The `scanf()` function takes a text stream from the keyboard, extracts and formats the data from the stream according to a format control string and then stores the data in a specified program variables.**
- ✓ Syntax of the `scanf()` function:
`scanf ( "control string", arg1, arg2, arg3,..... argn);`
- ✓ Control string specifies the type and format of the data that has to be obtained from the keyboard and stored in the memory locations pointed by `arg1, arg2, ..., argn`, i.e., arguments are actually the variable addresses where each piece of data is to be stored.
- ✓ Prototype of the control string can be given as:
`%[*][width][modifier] type`
- ✓ Here the `*` is an optional argument which indicates that data should be read from the stream, but ignored (not stored in memory location).

Width

- ✓ This is an optional argument that specifies the maximum number of characters to be read.
- ✓ Fewer characters are read if `scanf()` encounters a white space and will stop processing further.

Modifier

- ✓ It is an optional argument that can be `h`, `l`, or `L` for the data pointed by corresponding additional arguments.
- ✓ Modifier `h` is used for short int or unsigned short int, `l` is used for long int, unsigned long int, or double values. Finally, `L` is used for long double data values.

Type

- ✓ It specifies the type of data that has to be read.
- ✓ The type specifiers for `scanf()` function are same as that of `printf()` function.
- ✓ The `scanf` function ignores any blank spaces, tabs and newlines entered by the user.
- ✓ The address of the variable is denoted by an `&` sign followed by the name of the variable.

Rules to use a scanf function:

Rule 1: The `scanf` function works until:

- the maximum number of characters has been processed
- a white space character is encountered,
- or an error is detected.

Rule 2: Every variable that has to be processed must have a conversion specification associated with it. Therefore, following scanf statement will generate an error as num3 has no conversion specification associated with it.

```
scanf(“ %d %d”, &num1, &num2, &num3);
```

Rule 3: There must be a variable address for each conversion specification. Therefore following scanf will generate an error as no variable address is given for the third conversion specification.

```
scanf(%d %d %d”, &num1, &num2);
```

Rule 4: An error will be generated if the format string is ended with the white space character.

Rule 5: The data entered by the user must match the character specified in the control string, otherwise, an error will be generated and scanf will stop its processing.

For example, consider the following scanf of statement

```
scanf(“%d / %d”, &num1, &num2);
```

Here the slash in the constant string is neither a white space character nor a part of the conversion specification, so the users must enter data of the form 21/46.

Rule 6: Input data values must be separated by spaces.

Rule 7: Any unread data value will be considered as a part of data input in the next call to the scanf.

Rule 8: When the field width specifier is used, it should be large enough to contain the input data size.

Examples of printf/scanf:

1. Code to input values in variables of different data types:

- int num;
scanf(“%d”,&num);
- float salary;
scanf(“%f”, &salary);
- char ch;
scanf(“%c”,&ch);
- char str[10];
scanf(“%s”, str);

2. Reading variables of different data types in one statement:

```
int num;  
float fnum;  
char ch;  
char str[10];  
scanf(“%d %f %c %s”, &num, &fnum, &ch, str);
```